

High-Level Design (HLD)

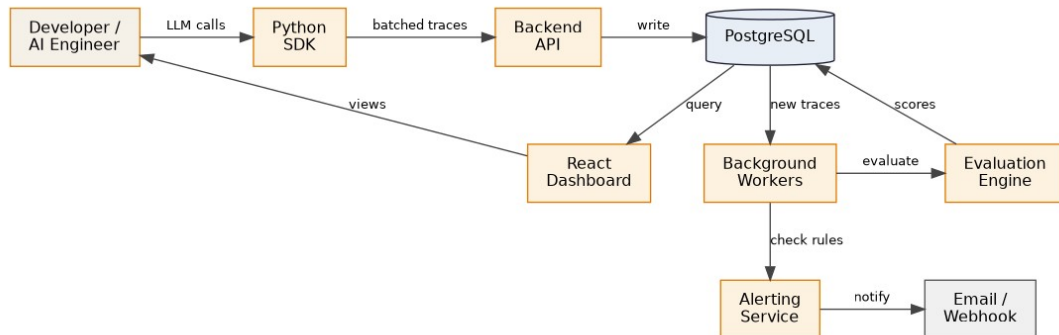
LLM Observability Platform

Polaris School of Technology
On-the-Job Training (OJT) Project
B.Tech CSE (AI/ML) — Batch 2025–2029
Date: September 2026

Project Title	LLM Observability Platform
Team Members	Shuban Mutagi, Sagar Halladakeri
Mentor	Tanmay
Track	AI/ML — Generative AI
Version	1.0

Architecture

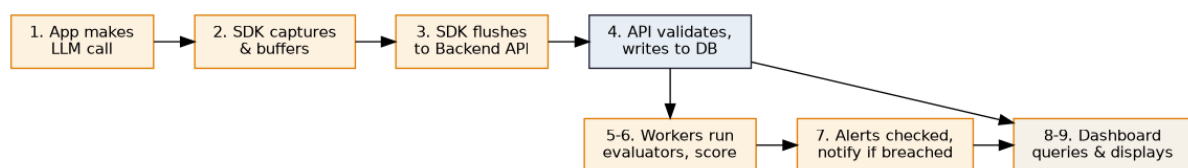
The platform follows a telemetry-pipeline architecture: an SDK captures LLM call data at the source, a backend API ingests and stores it, background workers run evaluations and alert checks, and a dashboard surfaces everything to the user.



Components

Component	Responsibility
Python SDK	Wraps LLM provider calls, captures telemetry (prompt, completion, tokens, latency, cost), batches and flushes to the backend asynchronously
Backend API	Validates and ingests trace data, serves query endpoints for the dashboard, handles auth and project scoping
PostgreSQL	Stores traces, spans, evaluations, alert rules, users, and projects
Background Workers	Run evaluation jobs (LLM-as-judge, heuristics) and check alert thresholds asynchronously
Evaluation Engine	Scores LLM outputs on relevance, faithfulness, safety, and format compliance
Alerting Service	Evaluates configured alert rules against incoming metrics and triggers notifications
React Dashboard	Trace Explorer, analytics charts, evaluation views, alert configuration, and project settings
Auth Service	User registration, login (JWT), role-based access control (Admin, Member, Viewer)
Notification Channels	Email and webhook delivery for triggered alerts

End-to-End Data Flow



1. Developer's application makes an LLM call, wrapped by the SDK.
2. SDK captures telemetry (prompt, completion, tokens, latency, cost) and buffers it locally.

3. SDK flushes buffered traces to the Backend API in batches, authenticated via the project's API key.
4. Backend API validates the payload, calculates cost from token counts, and writes the trace and its spans to PostgreSQL.
5. Background workers pick up new traces and run configured evaluators (LLM-as-judge, heuristics).
6. Evaluation results are written back to PostgreSQL, linked to their source span.
7. Background workers check active alert rules against recent metrics; a breach triggers a notification via email or webhook.
8. The React Dashboard queries the Backend API to render the Trace Explorer, analytics charts, and evaluation views.
9. User reviews traces, evaluation scores, and analytics, and adjusts alert rules or prompts as needed.

Scalability

Start as modular services rather than microservices. The ingestion path (SDK → API → DB) and the processing path (evaluation, alerting) have different runtime characteristics, so they are kept as separable components even within a single deployable backend initially.

- SDK batches and compresses traces client-side to reduce request volume.
- Trace ingestion is append-only and horizontally scalable behind the API layer.
- Background workers are decoupled from the request path — evaluation and alerting never block trace ingestion.
- Introduce a message queue (e.g., Redis/Celery broker) only when evaluation volume requires independently scalable workers.
- Dashboard queries are paginated and indexed by project, time range, and trace ID to keep response times flat as data grows.

Reliability

- SDK buffers traces locally and retries with backoff if the backend is unreachable — no silent data loss on the client side.
- Trace ingestion is idempotent — re-sent batches do not create duplicate traces.
- Evaluation and alerting failures do not affect trace storage — a trace is persisted regardless of whether evaluation succeeds.
- Failed evaluation jobs are retried a bounded number of times before being marked FAILED.
- Job status (PENDING, RUNNING, COMPLETED, FAILED) is tracked so failures are visible rather than silent.

Security

- Authentication via JWT for dashboard users; API keys for SDK-to-backend trace ingestion.
- Role-based access control: Admin, Member, Viewer, scoped per project.
- Traces are isolated per project — no cross-project data access.
- All traffic encrypted in transit (HTTPS).
- Secrets (API keys, DB credentials, LLM provider keys) stored in environment variables, never committed to source control.
- Prompt and completion text may contain sensitive data — stored for debugging but designed to support future redaction/masking policies.

End of HLD